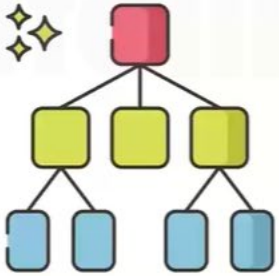
What are Modules?

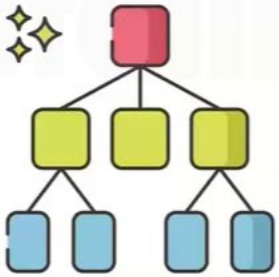
Whenever we execute some python code in an interpreter and then quit the python interpreter the definitions made in that interpreter session are lost



© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

What are Modules?

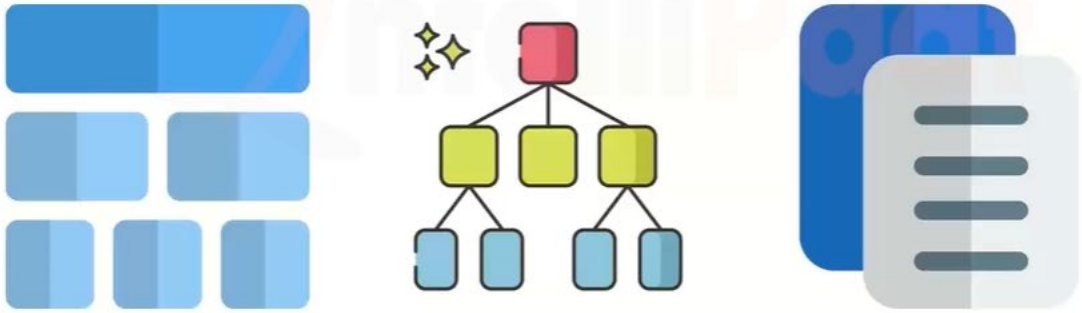
However, if we wish to save these definitions for later reuse you can save it in a file with a .py extension



© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

What are Modules?

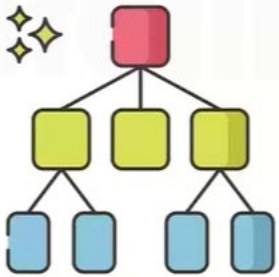
If this file gets really long then you can split this file into smaller files so that they can be managed easily



© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

What are Modules?

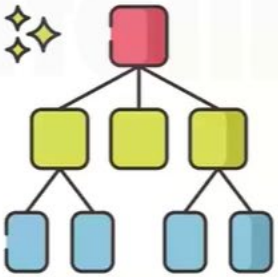
You might also want to group together a few functions in a file which can be used in several other projects



© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

What are Modules?

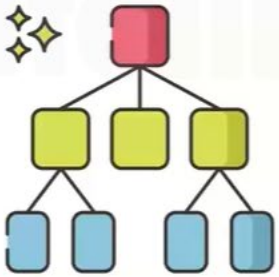
In Python we can do these things by saving it together in a file and then using it in other files or interpreter sessions



© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

What are Modules?

To put it simply, A python module is a python file that contains code which can be imported and used by other python script files



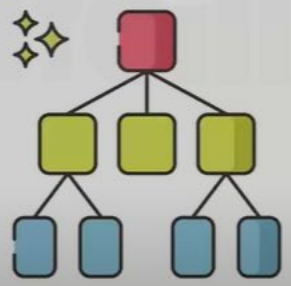
© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

lms.intellipaat.com

What are Modules?

IntelliPaat

We can import a variable, a function and a class from a module



Why use Modules?

Modules provide several benefits such as:

Simple

Reusable

Scoped

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Why use Modules?

Modules allow us to simplify our code to a great extent

Simple

Reusable

Scoped

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Why use Modules?

Each module can focuses on a small portion of a problem

Simple

Reusable

Scoped

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Why use Modules?

It makes code easier to understand and write

Simple

Reusable

Scoped

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Why use Modules?

A module can contain code which can later be used in other files

Simple

Reusable

Scoped

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Why use Modules?

One module can be imported and used by multiple files and all these files will be using the same code

Simple

Reusable

Scoped

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Why use Modules?

A module can also be shared and used in other python projects if the need be

Simple

Reusable

Scoped

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Why use Modules?

Every bit of code in a Python module is scoped to that file

Simple

Reusable

Scoped

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Why use Modules?

Which means that two modules containing functions with same name can co-exist in a projects

Simple

Reusable

Scoped

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Why use Modules?

However, it is possible that these two functions are imported in the same file which may cause naming conflict

Simple

Reusable

Scoped

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

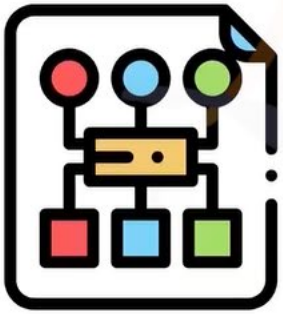
How to use Modules?

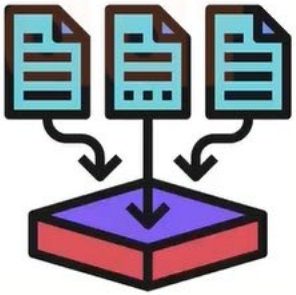
To make use of modules we first need to create a module

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

How to use Modules?

Creating a module is quite easy, all you have to do put all the code you wish to be in a module in a file

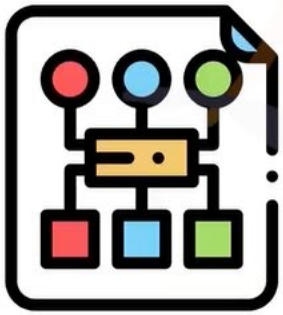


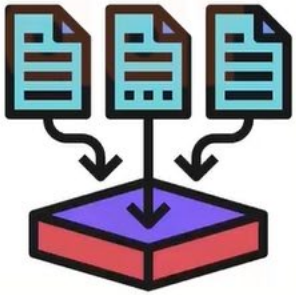


© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

How to use Modules?

Make sure that that file has a .py extension, otherwise it will not be imported by python

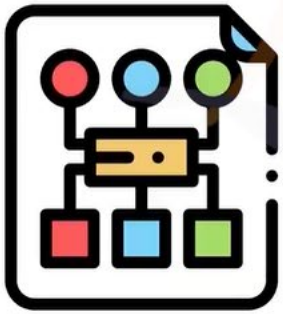


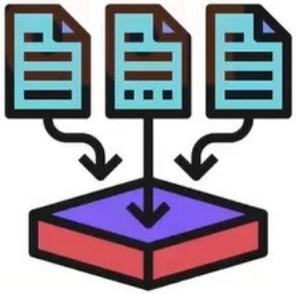


© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

How to use Modules?

Now, this file is a module and can be used by other python files

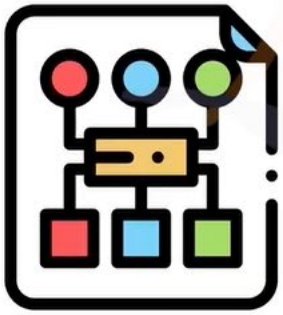


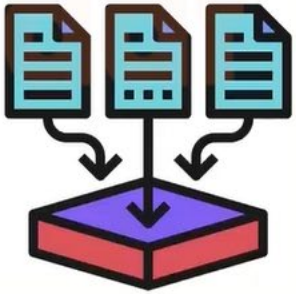


© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

How to use Modules?

To make use of this module we have to import items from this module which we wish to use

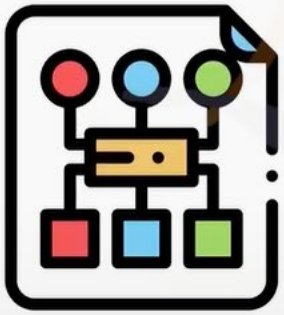


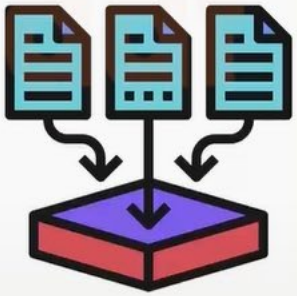


© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

How to use Modules?

For this we make use of the import keyword in python, which can be done in several ways





© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Importing from Modules

Using this way we can import an entire module which has the name set to sound

import sound

import sound as sound_module

from module import item

from sound import *

from sound import echo as echo_sound

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Importing from Modules

Everything inside the sound module can be accessed by using the name sound: e.g. sound.play(), sound.version etc.

```
import sound
```

```
import sound as sound_module
```

```
from module import item
```

```
from sound import *
```

```
from sound import echo as echo_sound
```

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Importing from Modules

Using this way we can import module with the name of sound and change its name to sound_module

- import sound
- import sound as sound_module
- from module import item
- from sound import *

Importing from Modules

This is done so that two modules with the same name can be used inside a file

import sound

import sound as sound_module

from module import item

from sound import *

from sound import echo as echo_sound

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Importing from Modules

Using this syntax we can import a specific item from a module instead of importing the whole module

import sound

import sound as sound_module

from module import item

from sound import *

from sound import echo as echo_sound

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Importing from Modules

This is useful as it increases the speed with which a module will be loaded as we are inly importing specific items from that module instead of everything

```
import sound
```

```
import sound as sound_module
```

```
from module import item
```

```
from sound import *
```

Importing from Modules

Using this syntax we can import everything from a module into the current namespace

import sound

import sound as sound_module

from module import item

from sound import *

from sound import echo as echo_sound

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Importing from Modules

So if sound module contains a method named echo, we can now use it directly like this: echo

import sound

import sound as sound_module

from module import item

from sound import *

from sound import echo as echo_sound

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Importing from Modules

Usually importing everything from a module is discouraged as it makes our code more ambiguous

```
import sound
```

```
import sound as sound_module
```

```
from module import item
```

```
from sound import *
```

```
from sound import echo as echo_sound
```

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Importing from Modules

Using this syntax we can use import a specific item from a module and give it a different name

import sound

import sound as sound_module

from module import item

from sound import *

from sound import echo as echo_sound

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Importing from Modules

This is useful when we are importing items form different modules and these items have the same name

```
import sound
```

```
import sound as sound_module
```

```
from module import item
```

```
from sound import *
```

Executing Modules as Scripts

An important thing to note when importing a module is that when you do so, python interpreter executes any bit of code inside the module

```
if __name__ == "__main__":
    print("Executin code in this module")
```

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Executing Modules as Scripts

When you write import sound, all the code inside this module is executed by the python interpreter first and then imported in the current module

```
if __name__ == "__main__":
    print("Executin code in this module")
```

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Executing Modules as Scripts

So, if you wish to make this file useable as a script which can be executed and a module from which functionality can be imported, then we can use of the global __name__ property

```
if __name__ == "__main__":
    print("Executin code in this module")
```

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Executing Modules as Scripts

The __name__ property is set to "__main__" when a script is executed using python interpreter, i.e. using the command python sound.py

```
if __name__ == "__main__":
    print("Executin code in this module")
```

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Executing Modules as Scripts

So if there is some code which should be executed when this file is executed then you put it inside an if statement which checks if the __name__ property is set to "__main__"

```
if __name__ == "__main__":
    print("Executin code in this module")
```

© COPYRIGHT INTELLIPAAT.COM. ALL RIGHTS RESERVED.

Executing Modules as Scripts

This is a common practice to use a python file as an executable script as well as an importable module

```
if __name__ == "__main__":
    print("Executin code in this module")
```